IN432 - Advanced Web Technologies

Session 1



Dr. Zaher Yamak

Lebanese University — Master M1

Your starting point

Client	Server and data	Protocols	Workflow
HTML	PHP	HTTP	Git
CSS	Laravel	JSON	Python
JavaScript	SQL	REST	Flask
			Docker

0 Not yet familiar 1 Used it 2 Can explain a choice

By the end of today

Understand the map

Explain how browser, HTTP, backend, API and database cooperate in a modern web application.

Design a REST API

Translate a simple domain into resources, routes, methods, payloads and status codes.

Build with Flask

Create and run a minimal Flask API, then test it using HTTP requests.

Why “advanced web” is no longer only HTML + CSS + JavaScript + PhP

Modern web applications are distributed systems with user interfaces

A professional web app usually combines:

UI

Reusable components, state, routing, accessibility

API

Resources, validation, errors, security, versioning

Data

Modeling, persistence, transactions, queries

Ops

Git, containers, deployment, monitoring

AI

Assistive coding, review, docs, tests

Classical web

Full-stack product engineering

This course teaches the connections between these parts.
The language and framework are tools;
The architecture is the real skill.

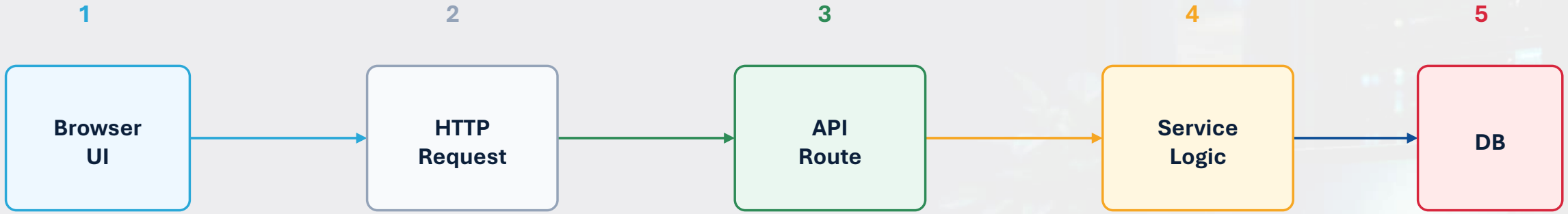
QUESTION

When you type a URL and press Enter, what really happens?



Answer: the request journey

A useful mental model for the entire course



- The browser is not “the web app”; it is one client of a system.
- HTTP is the contract between client and server.
- The API exposes capabilities; the database stores state.
- The response returns data, HTML, or an error that the client must handle.

Architecture vocabulary you must use precisely

Clear terms reduce confusion in full-stack teams

Client

Software that initiates requests: browser, mobile app, CLI, another service.

Server

Software that listens for requests and returns responses.

API (Application Programming Interface)

A contract that exposes operations and data in a controlled way.

Frontend

Client-side interface and user interaction logic.

Backend

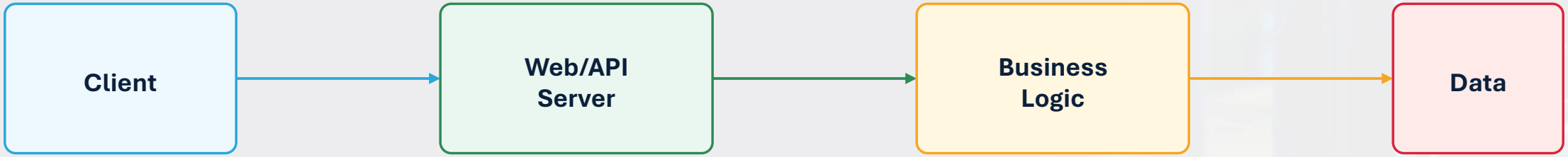
Server-side code: routes, services, data access, security.

Full-stack

A system that connects frontend, backend, database and deployment.

Client–server, three-tier, full-stack: three views of the same system

Architecture diagrams depend on the question being asked



Client–server view

Useful when explaining who asks and who answers.

Three-tier view

Useful when separating UI, logic and data responsibilities.

Full-stack view

Useful when building the complete product and deployment workflow.

Do not memorize diagrams; learn what each diagram helps you reason about.

Monolith vs Microservices

A design choice depends on team, domain and operational maturity

Modular monolith

One deployable application with clear internal modules. Easier to start, test and deploy. Good for most student projects and many real products.

Microservices

Several independently deployed services communicating over networks. Useful for large teams and independently scalable domains, but adds operational complexity.

Quick review: the three core client-side languages

We review only what is needed to connect to APIs later

HTML = structure

Meaning of content: forms, headings, inputs, links, semantic elements.

CSS = presentation

Layout, spacing, responsive design, states, visual consistency.

JavaScript = behavior

Events, DOM updates, fetch calls, validation, dynamic UI.

```
<form id="loginForm">
  <input name="email" type="email">
  <button>Login</button>
</form>

fetch('/api/login', { method: 'POST', body:
... })
```

- HTML gives the form its meaning.
- CSS makes it usable and responsive.
- JavaScript connects user action to the backend API.

Quick review: the core server-side languages

We review only what is needed to connect to APIs later

Backend : PhP / Laravel

Routes and controllers, validation, and database access.

API : Python + Flask

Python routes and view functions for APIs. Our stack for the lab.

JavaScript + Node.js / Express

Node.js runs JavaScript on the server. Express adds routes and middleware.

```python
# Python / Flask
@app.get("/api/students/<int:student_id>")
def get_student(student_id):
    student = find_student(student_id)
    if student is None:
        return {"error": "Not found"}, 404
    return student, 200
```

- Routes map HTTP methods and URLs to server code.
- Server code validates input, applies rules and accesses data.
- The server returns a status code and HTML or JSON.

Other examples: Java / Spring Boot and C# / ASP.NET Core

Interactive check: identify the responsibility

Case 1

“The button is disabled until the email is valid.”

JavaScript

Case 2

“The user profile is saved after submission.”

Backend/PHP/...

Case 3

“The layout becomes one column on mobile.”

CSS

Case 4

“The endpoint refuses requests without a token.”

API

Case 5

“The title of the page is marked as a heading.”

HTML

PART 2

HTTP and REST



When Programs Are Separate

- Consider we have multiple programs (services).
- Example:
  - Login program
  - Product program
  - Payment program
- They run on different machines.
- Question:
  How can they talk to each other?

They Cannot Use Function Calls

- Inside one program we call a function:
  - checkPassword(user, pass)
- But across machines:
  - no shared memory
  - no direct function call
  - no same process
- We must send messages over the network.

Communication Over the Network

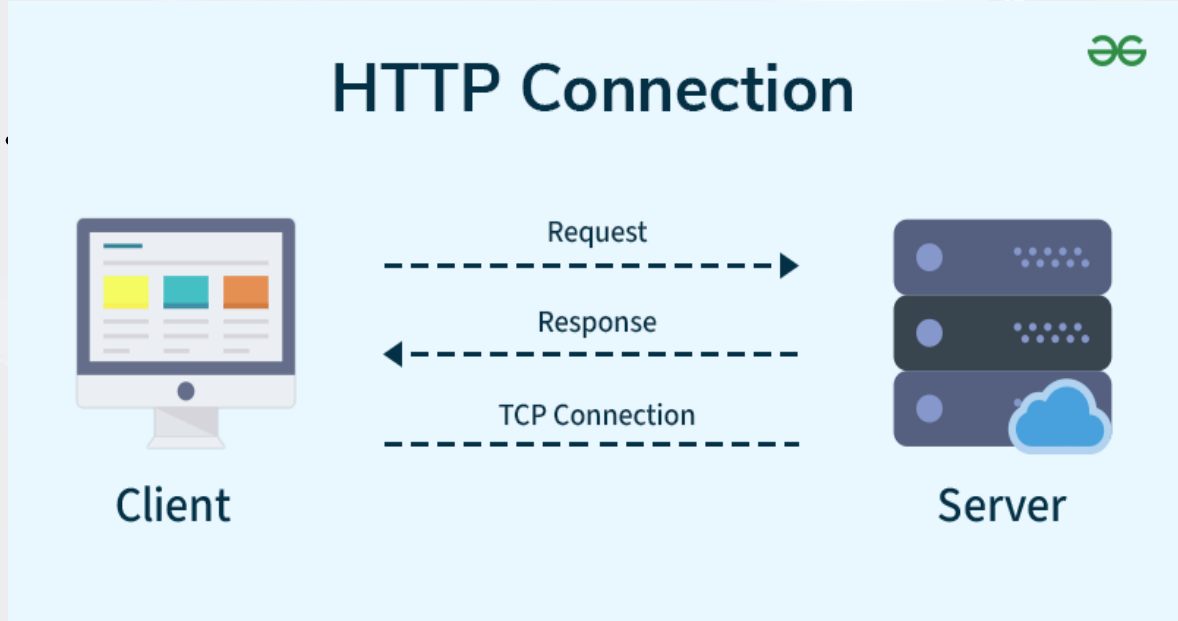
- To communicate, programs use:
  - Internet (or local network)
  - Requests
  - Responses
- One program asks → another program answers.
- This is similar to how a browser loads a webpage.

Request / Response Model

- Basic idea:
  - Client sends a request
    Server processes
    Server sends a response
- This pattern is everywhere in computing.

What Is HTTP?

- HTTP = communication language used on the web.
- When you open a website:
  - Your browser sends HTTP request
    Server returns HTTP response
- Services use the same mechanism.



Example of HTTP in Daily Life

- Typing in browser:
  - www.shop.com/products
- Browser sends:
  - “Give me the products page”
- Server replies with data.
- APIs work the same way — but send data instead of HTML.

What Is an API?

- API = rules that allow programs to communicate.
- It defines:
  - what can be requested
  - how to request it
  - what response looks like
- API is a contract between programs.

Simple API Example

- Product service exposes:
  - GET /products
- Another program calls it to obtain products.
- The caller does not know internal code.
  It only knows the API.

Endpoint Definition

- An endpoint is an address of a function on a server.
- Example:
  - /users
    /orders
    /payments
- Each endpoint represents an available action.

HTTP is a structured conversation

Every API call is a request followed by a response

Request

```
GET /api/tasks?status=open HTTP/1.1
Host: example.com
Accept: application/json
Authorization: Bearer <token>
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json

[{ "id": 1, "title": "Prepare lab", "done": false
}]
```

- Method: what the client wants to do.
- Path: the resource or collection.
- Headers: metadata such as format, authorization, caching.
- Body: optional data sent by the client.
- Status code: machine-readable result category.

HTTP methods: use verbs consistently

The method should express the operation; the URL should express the resource

Method	Typical meaning	Example
GET	Read data without changing server state	GET /api/tasks
POST	Create a new resource or trigger processing	POST /api/tasks
PUT	Replace an entire resource	PUT /api/tasks/12
PATCH	Partially update a resource	PATCH /api/tasks/12
DELETE	Remove a resource	DELETE /api/tasks/12

Status codes: make results clear to clients

A good API teaches clients how to react

2xx Success

200 OK, 201 Created, 204 No Content

3xx Redirect

Resource moved or alternative location

4xx Client error

400 Bad Request, 401 Unauthorized, 404 Not Found

5xx Server error

500 Internal Server Error; the server failed

Activity

Choose codes for 5 cases

Quick cases

- A task is created successfully.
- The task id does not exist.
- The JSON body is missing the title field.
- The user is not authenticated.
- The server crashes because of a bug.

Expected: 201, 404, 400, 401, 500

Data Format Problem

- Programs must agree on data format.
- We need a universal format:
  - readable
  - structured
  - easy to parse
- This is where JSON appears.

What Is JSON?

- JSON is a text data format.
- Example:

```
{
 "id": 1,
 "name": "Phone",
 "price": 500
}
```

- Easy for humans and machines.

Why JSON Is Popular

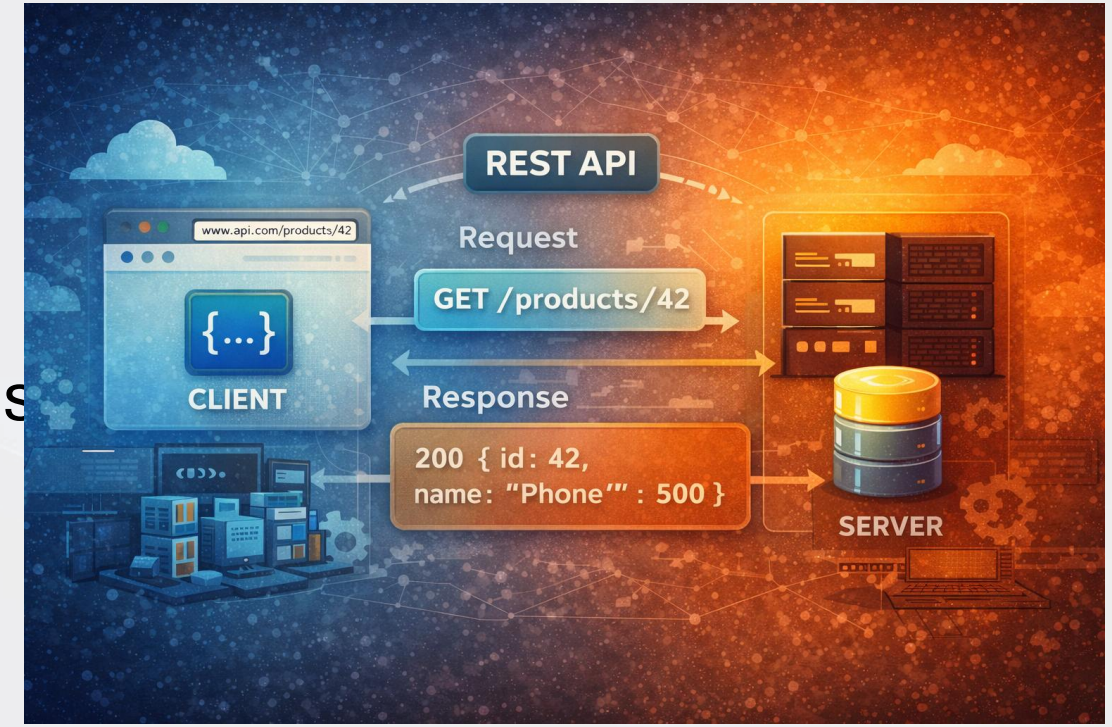
- JSON advantages:
  - simple
  - lightweight
  - supported everywhere
  - readable
- Most modern APIs use JSON.

Full API Example

- Client sends:
  - GET /products/1
- Server returns:
  - 200 OK
    { "id":1, "name":"Phone", "price":500 }
- Communication completed.

Now We Can Define REST

- REST (REpresentational State Transfer) = a way to design APIs using:
  - HTTP
  - URLs
  - JSON
- standard methods
- REST makes APIs predictable and s



REST Resource Concept

- REST organizes system around resources.
- Resources:
  - users
    products
    orders
- Each resource has its own URL.

REST Endpoints Example

- /products
  /products/10
  /orders
  /users/3
- URLs represent data, not actions.

CRUD mapping: one resource, several operations

The API contract should be easy to predict

Action	HTTP	Route	Input	Output
List tasks	GET	/api/tasks	Query filters	Array of tasks
Read one task	GET	/api/tasks/<id>	Path id	Task object or 404
Create task	POST	/api/tasks	JSON body	Created task + 201
Update task	PATCH	/api/tasks/<id>	Partial JSON body	Updated task or 404
Delete task	DELETE	/api/tasks/<id>	Path id	204 or 404

REST Operations Mapping (CRUD) : Example

- GET /products → list
  GET /products/10 → read one
  POST /products → create
  PUT /products/10 → update
  DELETE /products/10 → delete
- Clear and consistent structure.

API design anti-patterns to avoid

Small choices become expensive when the API grows

Verbs everywhere

/getAllTasks, /deleteTask, /addTask instead of resource routes.

One endpoint does everything

/api/action with a body field { action: "delete" }.

No error structure

Returning plain text or random error formats.

Always 200 OK

Client must guess whether the operation succeeded.

Leaking internals

Exposing stack traces, database details, or secret config.

No validation

Assuming client data is always correct.

Better API design makes frontend work easier and reduces hidden bugs.

PART 3

Backend foundations with Flask



What does the backend actually do?

A backend is not only a place where routes are written

Expose API routes

Define endpoints, methods, payloads and response formats.

Apply business rules

Check permissions, workflows, constraints and domain logic.

Manage data access

Read and write from databases safely and efficiently.

Validate input

Reject malformed or invalid requests before processing.

Handle errors

Return consistent status codes and error messages.

Protect the app

Authentication, authorization, rate limits, secure configuration.

Development environment: minimum setup

Before coding, every student must have a repeatable setup

```
# 1) Create a project folder
mkdir awt-session1 && cd awt-session1

# 2) Create and activate a virtual environment
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate

# 3) Install Flask
pip install Flask

# 4) Save dependencies
pip freeze > requirements.txt
```

Explain virtual environments

Each project keeps its own Python packages and versions.

Explain requirements.txt

A reproducible list of packages for teammates and deployment.

Do not commit .venv

Commit source code and requirements; exclude local environments.

Minimal Flask API: first working endpoint

The simplest possible “server answers client” example

```
# app.py
from flask import Flask, jsonify

app = Flask(__name__)

@app.get('/api/health')
def health():
    return jsonify({
        'status': 'ok',
        'service': 'advanced-web-api'
    }), 200

if __name__ == '__main__':
    app.run(debug=True)
```

- Flask creates the application object.
- @app.get registers a route and an HTTP method.
- jsonify creates a JSON response.
- Returning (body, status) makes the status explicit.
- debug=True is convenient in development, not production.

```
python app.py
# then open: http://127.0.0.1:5000/api/health
```

Routing and path parameters

Routes map HTTP requests to Python functions

```
@app.get('/api/tasks/<int:task_id>')
def get_task(task_id):
    # task_id is converted to int
    task = find_task(task_id)
    if task is None:
        return jsonify({'error': 'Task not found'}), 404
    return jsonify(task), 200
```

Path parameter

Part of the route that identifies a specific resource.

Converter

<int:task_id> rejects non-integer values before the function runs.

Student question: Which is better?

GET /api/tasks/12 or GET /api/tasks?id=12

Expected answer: both are possible, but /api/tasks/12 is the usual resource-item route.